Multi-Sensor Expansion Using FreeRTOS

Concurrent System Design for Real-Time Healthcare Monitoring

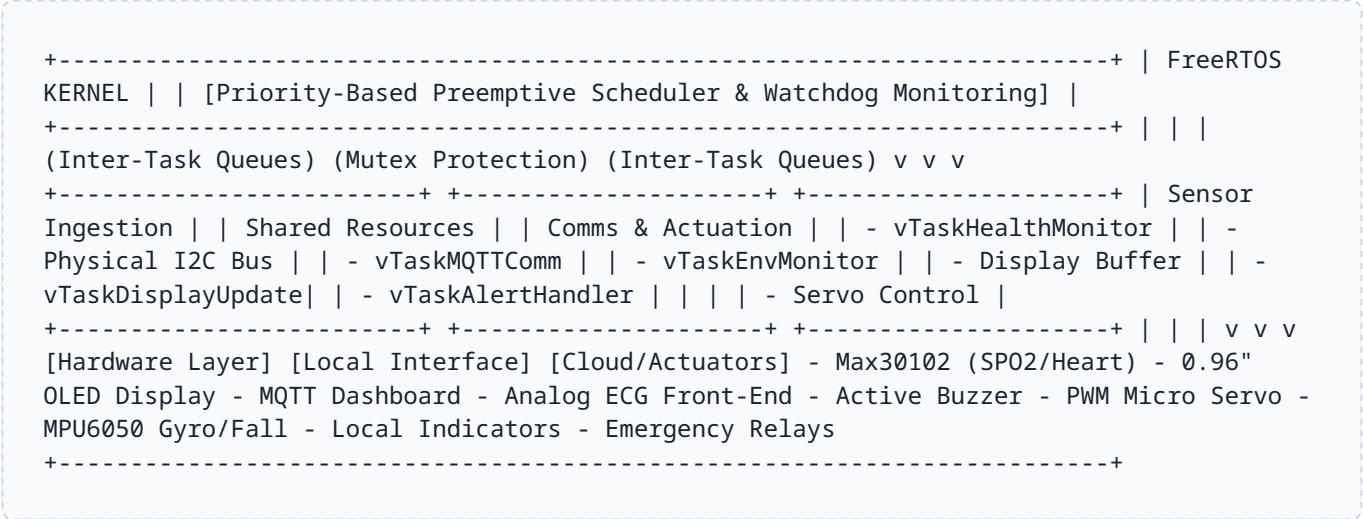
Institution: Padmabhooshan Vasantraodada Patil Institute of Technology (PVPIT), Budhgaon, Sangli

Project Track: Embedded Systems & Real-Time Operating Systems (RTOS)

Document: Technical Evaluation & System Architecture Report

1. System Architecture Diagram

The system utilizes an ESP32 microcontroller as the core computing engine managed by the preemptive FreeRTOS kernel. It establishes a multi-layered structure bridging hardware abstraction layers (biomedical and environmental sensors) with remote IoT cloud visualization frameworks over safe MQTT channels.



2. FreeRTOS Task Structure Explanation

To guarantee deterministic execution and avoid starvation of time-critical operations, tasks are structured with explicit execution priorities. Synchronization primitives eliminate race conditions across shared hardware channels.

Task Allocation Framework

Task Name	Priority	Periodicity / Trigger	Core Operational Responsibility
vTaskHealthMonitor	High (4)	10ms (Periodic)	Samples time-critical physiological signals (ECG data stream, Blood Pressure tracking).
vTaskEnvMonitor	Medium (2)	1000ms (Periodic)	Monitors local ambient data (Temperature, Humidity metrics).
vTaskAlertHandler	High (5)	Event-Driven	Processes anomalous threshold breaches; instantly triggers local buzzers.
vTaskMQTTComm	Medium (3)	500ms (Periodic/ Event)	Maintains handshake, updates telemetry feeds, and processes inbound cloud tokens.
vTaskDisplayUpdate	Low (1)	250ms (Periodic)	Refreshes the local 0.96" OLED screen with current health diagnostics.
vTaskWatchdog	Highest (6)	100ms (Periodic)	Tracks subsystem heartbeats and execution flags; kicks the hardware WDT.

Inter-Task Communication & Synchronization

- **Queues (`QueueHandle_t`):** Used to pass safe, thread-contained structure matrices containing vital health readouts from ingestion tasks straight to the transmission queues.
- **Mutexes (`SemaphoreHandle_t`):** Guards the single physical I2C transaction bus, blocking simultaneous entry from the display refreshing loops and the sensory data-pull pipelines.
- **Binary Semaphores:** Leveraged by the `vTaskAlertHandler` to unblock instantly upon receiving an external hardware interrupt flag issued by safety critical parameters.

3. Feed Mapping Table

MQTT topics are strict, streamlined pathways utilizing clean data payloads to achieve optimized latency and minimal packet footprints on network configurations.

Feed / MQTT Topic Name	Data Type	Direction	Description / Payload Context
`/health/ecg`	Float	Publish	Streamed high-frequency raw/filtered microvolt values for heart monitoring.
`/health/bp`	Integer	Publish	Systolic and Diastolic pressure estimations calculated locally (mmHg).
`/env/temp`	Float	Publish	Ambient environmental temperature tracking in degrees Celsius.
`/dosage/adjust`	JSON / Int	Subscribe	Remote command payload dictating medication dosage volume or angle modifications.
`/sys/status`	String	Publish	System heartbeat verification tags (e.g., "Online", "Offline Alert").

4. Dashboard Interface Layout

The monitoring station provides a secure graphical console containing analytical gauges, chart elements, and dynamic control widgets.

[Dashboard Visual Interface Console View]

• Widget 1: Real-time Line Chart tracking continuous incoming raw ECG waves.

• Widget 2: Dual Radial Gauges showing Blood Pressure boundaries (Systolic / Diastolic limits).

• Widget 3: Dynamic numeric readouts indicating Ambient Room Temperature and micro-hardware diagnostics.

• Widget 4: Interactive Input Field slider triggering direct physical PWM angle updates to topic "/dosage/adjust".

5. Remote Dosage Adjustment Logic

The adjustment routine operates via a strict, closed-loop processing paradigm inside the execution threads:

- **Subscription Interception:** The dedicated network service task `vTaskMQTTComm` maintains an active subscription hook targeting the `/dosage/adjust` feed.
- **Payload Boundary Verification:** Upon intercepting an update token, the parsing sequence isolates the target variable and passes it through an automated sanity checker ($0 \le Dosage \le 100$ percent) to eliminate hazardous boundary input anomalies.
- **Safe Task Signalling:** The confirmed value updates a shared data container wrapped tightly within an exclusive access Mutex lock block, preventing race conditions.

- **Physical Actuation Deployment:** The micro servo execution driver detects the structural shift, triggering a physical PWM profile update.

6. Servo Angle Mapping Explanation

To control mechanical valves or medical hardware setups using standard Pulse Width Modulation (PWM), abstract clinical adjustments are matched linearly onto an operational servo steering arc ($0°$ to $180°$).

The mathematical transformation translates the normalized dosage scale range $[0, 100\%]$ using a direct linear transfer function equation:

$$Angle = \left(\frac{\text{Target Dosage} - \text{Min Input}}{\text{Max Input} - \text{Min Input}} \times (\text{Max Angle} - \text{Min Angle}) \right) + \text{Min Angle}$$

Substituting our absolute hardware parameters configuration:

$$Angle = \left(\frac{\text{Target Dosage} - 0}{100 - 0} \times (180 - 0) \right) + 0 = \text{Target Dosage} \times 1.8$$

```
// Software Hardware-Bound Implementation Example
uint8_t calculate_servo_angle(int dosage_percent) {
    if (dosage_percent < 0)   dosage_percent = 0;
    if (dosage_percent > 100) dosage_percent = 100;

    return (uint8_t)(dosage_percent * 1.8);
}
```

7. Dynamic Sampling Implementation

To conserve network bandwidth and local resource allocations while maintaining clinical safety standards, the system incorporates a context-aware sampling algorithm:

- **Baseline Nominal State:** When vital signals fit safely within safe, standardized thresholds, the processing cycle operates in a low-power mode, sampling environmental and biomedical parameters at extended intervals (e.g., 10-second sleep cycles).
- **Emergency Real-Time Transition:** Upon detection of anomalous inputs (such as an acute heart rate surge or impact acceleration registered via the MPU6050 fall detection module), an immediate state transition occurs.
- **FreeRTOS Primitive Manipulation:** The `vTaskAlertHandler` forces an instant change in the blocking parameters of the primary reading task using FreeRTOS tracking primitives:

```
xTaskDelayUntil(&xLastWakeTime, xHighFidelityPeriod);
```

This forces the task into a high-frequency real-time recording loop, generating fine-grained signal curves required for remote emergency triage.

8. Offline Detection & Safe Fallback Mechanisms

Network connectivity fluctuations are mitigated via dual-ended architectural safety loops to prevent system locks or data loss during clinical emergencies.

1. Cloud-Side Monitoring (MQTT Last Will and Testament)

During the initial network protocol authorization sequence, `vTaskMQTTComm` configures a **Last Will and Testament (LWT)** meta-packet. The broker retains this message block natively. If the system drops connection unexpectedly due to hardware damage or power loss, the cloud broker automatically fires the packet:

- **LWT Broadcast Feed:** `/sys/status`
- **LWT Target Payload:** `"Offline Unsafely"`

2. Device-Side Fail-Safe Operation

If network connection packets fail repeatedly, the device increments an internal error tracker. Once a threshold is crossed, the system switches to **Local Cache Mode**: historical biomedical metrics bypass the active transmission queues and are written to local non-volatile storage flash memory blocks. Upon reconnection validation, the device safely flushes the historical records back to the remote server, maintaining an unbroken medical record.